Pseudocode

Compiled by Zuhaib Abdul Khalique

Q1) Write a pseudocode to input the marks of 3 subject, calculate and output Total marks & Average Marks

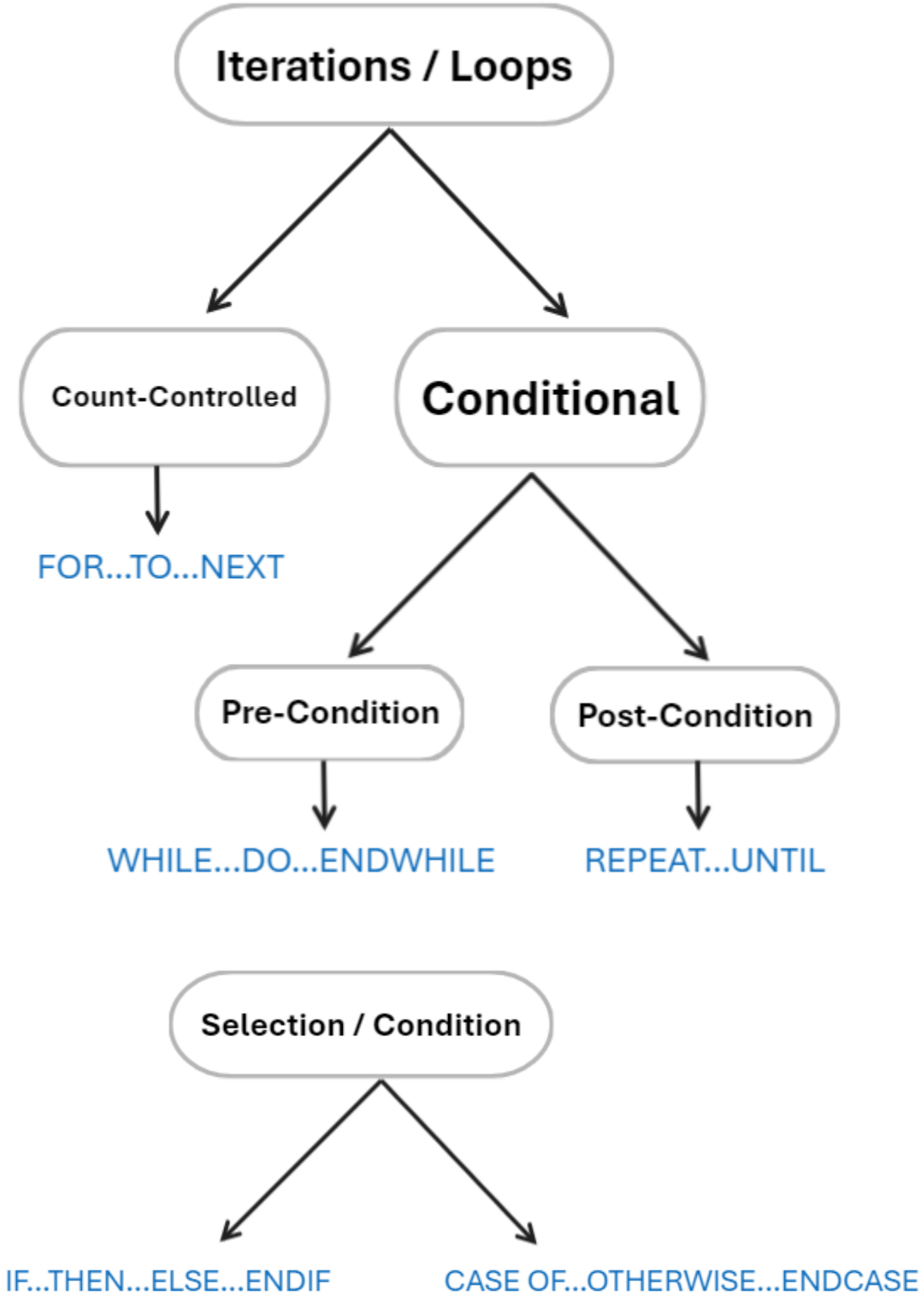
```
Total ← 0
Avg ← 0
FOR COUNT ← 1 TO 3
        OUTPUT "Enter Marks of Subject ", COUNT
        INPUT Marks
        Total ← Total + Marks
NEXT
Avg ← Total/3
OUTPUT "Total of 3 subjects marks is: ", Total
OUTPUT "Average of 3 subjects marks is: ", Avg
```

Q2) Write a pseudocode to input percentage of a student, output 'Pass' if percentage is 60 and above otherwise output fail

```
INPUT Percentage
IF Percentage >= 60 THEN
        OUTPUT "Pass"
ELSE
        OUTPUT "Fail"
ENDIF
```

Q3) Write a pseudocode to input percentage, output 'A' if percentage is 70 and above and less than equal to 100, output 'B' if percentage is 50 and above but below 70, output 'C' if percentage is 30 and above but below 50, output 'U' if percentage is 0 and above but below 30, otherwise output invalid percentage

```
INPUT Percentage
IF Percentage >= 70 AND Percentage <= 100 THEN
        OUTPUT "A"
ELSE IF Percentage >= 50 AND Percentage < 70 THEN
        OUTPUT "B"
ELSE IF Percentage >= 30 AND Percentage < 50 THEN
        OUTPUT "C"
ELSE IF Percentage >= 0 AND Percentage < 30 THEN
        OUTPUT "U"
ELSE
        OUTPUT "Invalid Percentage"
ENDIF
```



- **If we know the no. of iterations then count-controlled loop is the best practice**

- **by default count controlled loop has a step of +1**

Q4) Write a pseudocode to input two integer values (assume both will be different) output the largest one

```
INPUT N1
INPUT N2
IF N1 > N2
        OUTPUT N1
ELSE
        OUTPUT N2
ENDIF
```

Q5) Write a pseudocode to input three integer values assume all three will be different. Output the largest one

```
INPUT N1
INPUT N2
INPUT N3
IF N1 > N2 AND N1 > N3 THEN
        OUTPUT N1
ELSE IF N2 > N1 AND N2 > N3 THEN
        OUTPUT N2
ELSE
        OUTPUT N3
ENDIF
```

Q6) Write a pseudocode to output your name five times without using loop

```
OUTPUT "Zuhaib"
OUTPUT "Zuhaib"
OUTPUT "Zuhaib"
OUTPUT "Zuhaib"
OUTPUT "Zuhaib"
```

Using Loop:

```
FOR INDEX ← 1 TO 5
        OUTPUT "Zuhaib"
NEXT INDEX
```

Index	Output
1	Zuhaib
2	Zuhaib
3	Zuhaib
4	Zuhaib

5	Zuhaib

Q7) Write a pseudocode to print the sequence of numbers from 1 till 10

```
FOR i ← 1 TO 10
        OUTPUT i
NEXT i
```

Q8) Write a pseudocode to output all the even numbers from 1 till 10

```
FOR j ← 1 TO 10
        IF MOD(j, 2) = 0 THEN
                OUTPUT j
        ENDIF
NEXT j
```

OR

```
FOR j ← 1 TO 10 STEP 2
        OUTPUT j
NEXT j
```

Q9) Write a pseudocode to output all the odd numbers from 1 till 10

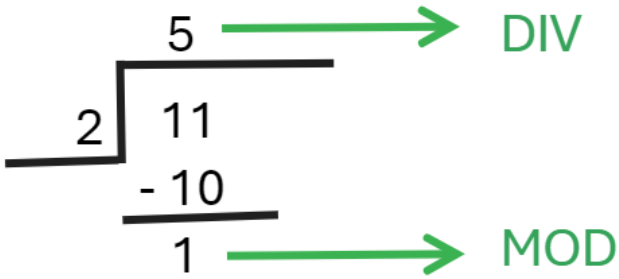
```
FOR COUNT ← 1 TO 9 STEP 2
        OUTPUT COUNT
NEXT COUNT
```

Q10) Write a pseudocode to putput sequence of numbers from 1 till 10 in reverse order

```
FOR J ← 10 TO 1 STEP -1
        OUTPUT J
NEXT J
```

X ← 11 MOD 2
Y ← 11 DIV 2

RESULT → X = 1 & Y =5



Operators

+	AND
-	OR
*	NOT
/	MOD
>	DIV
<	
>=	
<=	
<>	

Q11) Write a pseudocode to input 10 integer values from user. Find out an output of the number of these values that are even or odd

```
Even ← 0
Odd ← 0
FOR INDEX ← 1 TO 10
        INPUT Num
        IF Num MOD 2 = 1 THEN
                Odd ← Odd +1
        ELSE
                Even ← Even + 1
        ENDIF
NEXT INDEX
OUTPUT "The number of even numbers are: ", Even
OUTPUT "The number of odd numbers are: ", Odd
```

Q12) Write a pseudocode which allows user to input a number. Find out an output of the factorial of that entered number

```
Value ← 1
INPUT Num
IF Num < 0 THEN
        OUTPUT "Negative Value hence factorial is not defined"
ELSE
        WHILE Num <> 0 DO
                Value ← Value * Num
                Num ← Num – 1
        ENDWHILE
        OUTPUT "Factorial of entered value is: ", Value
ENDIF
```

OR

```
Value ← 1
INPUT Num
IF Num >= 0 THEN
        IF Num > 0 THEN
                FOR INDEX ← Num TO 1 STEP -1
                        Value ← Value * Num
                NEXT INDEX
        ENDIF
        OUTPUT "Factorial of entered value is: ", Value
ELSE IF Num < 0 THEN
        OUTPUT "Factorial is not defined for negative numbers"
```

OR (Recursive Approach using a function)

```
FUNCTION Factorial(ThisNum : INTEGER) RETURNS INTEGER
        IF ThisNum > 1 THEN
                RETURN ThisNum * Factorial(ThisNum – 1)
        ELSE
                IF ThisNum = 1 OR ThisNum = 0 THEN
                        RETURN 1
                ELSE
                        RETURN -1
                ENDIF
        ENDIF
ENDFUNCTION
```

Q13) Write a pseudocode which allows user to input two integer values. Calculate an output of the sum of all the values from first number to second number

```
INPUT Num1
INPUT Num2
Sum ← 0
IF Num1 < Num2 THEN
        FOR INDEX ← Num1 TO Num2
                Sum ← Sum + INDEX
        NEXT INDEX
ELSE
        FOR INDEX ← Num2 TO Num1
                Sum ← Sum + INDEX
        NEXT INDEX
ENDIF
OUTPUT "Sum of all values between two numbers including them is: ", Sum
```

Q14) Write a pseudocode to input two integer values variable N1 & N2. Swap the values and output them

```
INPUT N1
INPUT N2
Temp ← N1
N1 ← N2
N2 ← Temp
OUTPUT N1
OUTPUT N2
```

Question No. 6

An algorithm will:

- output **each** integer value between 100 and 200 that ends with the digit 7, for example, 107
- output a final count of the number of values that are output.

Write pseudocode for this algorithm.

Any variables used must be declared.

```
DECLARE Count : INTEGER
Count ← 0
FOR ← INDEX 107 TO 197 STEP 10
        OUTPUT INDEX
        Count ← Count + 1
NEXT INDEX
OUTPUT Count
```

OR

```
DECLARE Count : INTEGER
Count ← 0
FOR INDEX ← 100 TO 200
        IF INDEX MOD 10 = 7 THEN
                OUTPUT INDEX
                Count ← Count + 1
        ENDIF
NEXT INDEX
OUTPUT COUNT
```

Q15) A program will:

- input 50 unique integer values
- output the largest value
- output the average of the values **excluding** the largest value

Write a pseudocode to represent the algorithm

Variable declarations are not required

It is not necessary to check that each input value is unique

```
Total ← 0
INPUT N1
Large ← N1
Total ← Total + N1
FOR Count ← 1 TO 49
        INPUT Num
        Total ← Total + Num
        IF Num > Large THEN Large ← Num
NEXT COUNT
OUTPUT Large
Average ← (Total – Large) / 49
OUTPUT Average
```

OR (also checking if input is unique on next page)

Solution 2:

```
DECLARE Numbers : ARRAY[1:50] OF INTEGER
INPUT Numbers[1]
Total ← Numbers[1]
Large ← Numbers[1]
Unique ← True
FOR Count ← 2 TO 50
        REPEAT
                Unique ← True
                INPUT TempValue
                FOR Index ← 1 TO COUNT - 1
                        IF TempValue = Numbers[Index] THEN
                                Unique ← False
                                OUTPUT "Please enter a unique value!"
                NEXT Index
        UNTIL Unique
        Numbers[Count] ← TempValue
        Total ← Total + TempValue
        IF TempValue > Large THEN Large ← TempValue
NEXT Count

OUTPUT Large
Average ← (Total – Large) / 49
OUTPUT Average
```

PRE-CONDITION

```
Index ← 0
WHILE Index < 5 DO
        OUTPUT "Zuhaib"
        Index ← Index + 1
ENDWHILE
```

Index	Output
0	Zuhaib
1	Zuhaib
2	Zuhaib
3	Zuhaib
4	Zuhaib
5	

POST-CONDITION

```
Index ← 0
REPEAT
        OUTPUT "Zuhaib"
        Index ← Index + 1
UNTIL Index = 5
```

Index	Output
0	Zuhaib
1	Zuhaib
2	Zuhaib
3	Zuhaib
4	Zuhaib
5	Zuhaib

- Post condition loop always runs once. It first runs then check the condition. If condition is false it runs again otherwise it ends
- Pre condition loop only runs for the given condition to be true. It is like if statements iteration.

Q16) An algorithm is needed to process a sequence of numbers. Numbers may be positive or negative and may be integer or decimal.

The pseudocode will:

- prompt and input one number at a time until the value zero is input
- sum the negative numbers
- sum the positive numbers
- when zero is input, output the two sum values and then end.

Write the pseudocode needed. **(Reference Paper 2 Algorithm Design & Problem Solving Q7)**

```
DECLARE Num : REAL
PosSum ← 0
NegSum ← 0
INPUT Num
WHILE Num <> 0 DO
        IF Num > 0 THEN
                PosSum ← PosSum + Num
        ELSE IF Num < 0 THEN
                NegSum ← NegSum + Num
        ENDIF
        INPUT Num
ENDWHILE
OUTPUT PosSum
OUTPUT NegSum
```

OR

```
DECLARE Num : REAL
SumN ← 0
SumP ← 0
REPEAT
        INPUT Num
        IF Num > 0 THEN
                SumP ← SumP + Num
        ELSE IF Num < 0 THEN
                SumN ← SumN + Num
        ENDIF
UNTIL Num = 0
OUTPUT SumP
OUTPUT SumN
```

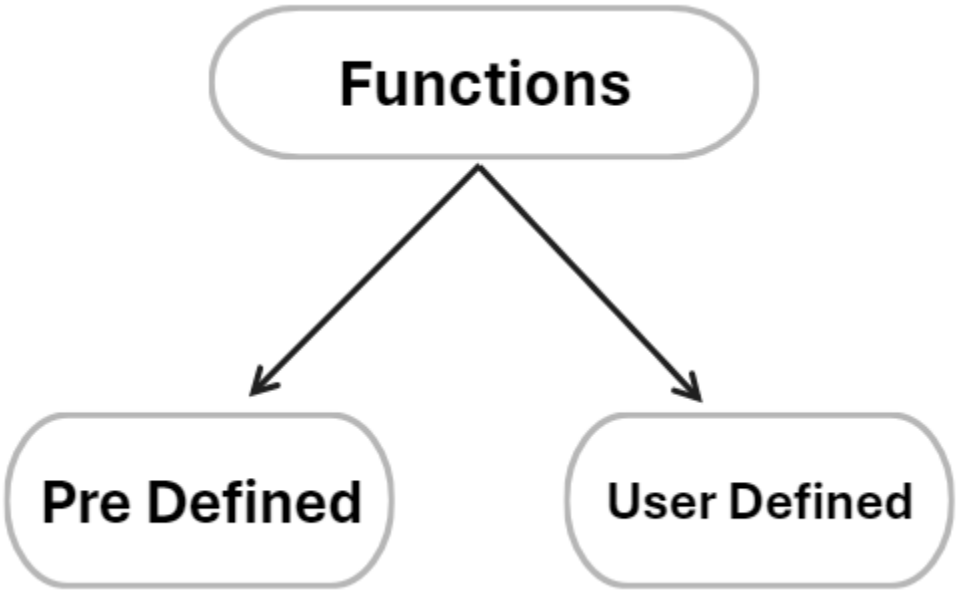
Q17) An algorithm will:

1. input an integer value
2. jump to step 6 if the value is zero
3. sum and count the positive values
4. sum and count the negative values
5. repeat from step 1
6. output the two sum values and the two count values.

Write a pseudocode on the following page to represent the algorithm.

Note that variable declarations are not required in the pseudocode.

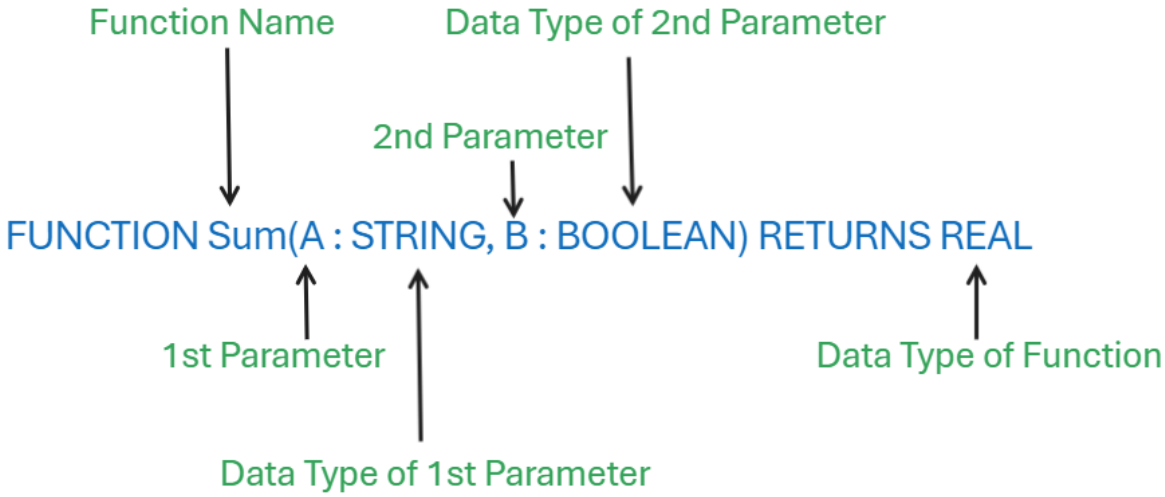
```
DECLARE Num : INETEGER
PosSum ← 0
NegSum ← 0
PosCount ← 0
NegCount ← 0
INPUT Num
WHILE Num <> 0 DO
        IF Num > 0 THEN
                PosSum ← PosSum + Num
                PosCount ← PosCount +1
        ELSE
                NegSum ← NegSum + Num
                NegCount ← NegCount +1
        ENDIF
        INPUT Num
ENDWHILE
OUPUT PosCount, PosSum
OUTPUT NegCount, NegSum
```



Library Functions
These functions are the **built-in functions** i.e., they are **predefined** in the library of the C. These are used to perform the most common operations like calculations, updation, etc. Some of the library functions are **printf, scanf, sqrt, etc**. To use these functions in the program the user has to use a **header file** associated with the corresponding function in the program.

User-defined Functions
These functions are designed by the user when they are writing any program because for every task we do not have a library of functions where their definitions are predefined. To perform according to the **requirement of the user**, the user has to develop some functions by itself, these functions are called **user-defined functions**. For such functions, the users can write their own logic according to the requirement.

- Using Pre Defined functions is easy and less time consuming and any errors are avoided that the programmer may make while writing user defined functions. However, user defined functions are useful to meet special requirements.



```
MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
```
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns `"BCD"`

```
LENGTH(ThisString : STRING) RETURNS INTEGER
```
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns `10`

```
LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
```
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns `"ABC"`

```
RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING
```
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns `"FGH"`

Code:

Y ← LENGTH("Zuhaib Abdul Khalique")
OUTPUT Y

Output:

21

Code:

myData ← "Zuhaib"
Z ← LEFT(myData, 3)
OUTPUT Z

Output:

Zuh

Code:

myData ← "Zuhaib"
Z ← RIGHT(myData, 3)
OUTPUT Z

Output:

aib

Code:

myData ← "Zuhaib Abdul Khalique"
A ← MID(myData, 8, 3)
OUTPUT A

Output:

Abd

Code:

```
X ← "ALPHA COLLEGE"
Y ← RIGHT(LEFT(X, 5), 3)
OUTPUT Y
```

Output:

PHA

Code:

```
X ← "ALPHA COLLEGE"
Y ← "HIGH SCHOOL"
Z ← LEFT(X, 5) & " " & RIGHT(Y, 6)
OUPTUT Z
```

Output:

ALPHA SCHOOL

- & sign is used for string concatenation

Q18) Write a **pseudocode** to output the content of myData in the given format
myData ← "ZUHAIB"
Output:
Z
ZU
ZUH
ZUHA
ZUHAI
ZUHAIB

```
FOR X ← 1 TO LENGTH(myData)
        OUTPUT LEFT(myData, X)
NEXT X
```

Q19) Write a **pseudocode** to input a letter and find out an output of how many times that letter appears in myData

```
Count ← 0
INPUT Character
FOR X ← 1 TO LENGTH(myData)
        IF MID(myData, X, 1) = Character THEN
                Count ← Count + 1
        ENDIF
NEXT X
```

Q20) Write a **pseudocode** to output the content of myData in reverse order
for e.g: if myData = "ALPHA" then output "AHPLA"

```
DECLARE Y: STRING
FOR X ← LENGTH(myData) TO 1 STEP -1
        Y ← Y & MID(myData, X, 1)
NEXT X
OUTPUT Y
```

Q21) Write a **pseudocode** to remove * from myData which is "Z*U**HA*I*B"

```
myData ← "Z*U**HA*I*B"
X ← ""
FOR INDEX ← 1 TO LENGTH(myData)
        Y ← MID(myData, INDEX, 1)
        IF INDEX <> "*" THEN
                X ← X & Y
        ENDIF
NEXT INDEX
OUTPUT X
```

Q22) Write a **pseudocode** to separate StudentID, Name, Age and Gender from myData which is "MR1001ALI16M"

```
myData ← "MR1001ALI16M"
X ← LENGTH(myData)
StudentID ← LEFT(myData, 6)
Name ← MID(myData, 7, X-9)
Age ← LEFT(RIGHT(myData, 3), 2)
Gender ← RIGHT(myData, 1)
```

Q23) Write a **pseudocode** to declare and initialize a 1D array myList to store 10 integer values

```
DECLARE myList : ARRAY[1:10] OF INTEGER
DECLARE INDEX : INTEGER
FOR INDEX ← 1 TO 10
        myList[INDEX] ← 0
NEXT INDEX
```

Q24) Write a **pseudocode** to input a value to be searched in the array myList containing 10 integers, if found output the array index otherwise output data not found. **(Linear Search)**

```
DECLARE INDEX : INTEGER
DECLARE FOUND : BOOLEAN
DECLARE SearchNum: INTEGER
FOUND ← FALSE
INDEX ← 1
OUPUT "Enter the number to search:"
INPUT SearchNum
WHILE FOUND = FALSE AND INDEX <= 10 DO
        IF SearchNum = myList[INDEX] THEN
                FOUND ← TRUE
        ELSE
                INDEX ← INDEX + 1
        ENDIF
ENDWHILE
IF FOUND = TRUE THEN
    OUTPUT "Number found at index: ", INDEX
ELSE
    OUTPUT "Number not found!"
ENDIF
```

OR
(Repeat..UNTIL)

```
DECLARE INDEX : INTEGER
DECLARE FOUND : BOOLEAN
DECLARE SearchNum: INTEGER
FOUND ← FALSE
INDEX ← 1
OUPUT "Enter the number to search:"
INPUT SearchNum
REPEAT
        IF SearchNum = myList[INDEX] THEN
                FOUND ← TRUE
        ELSE
                INDEX ← INDEX + 1
        ENDIF
UNTIL FOUND OR INDEX > 10
IF FOUND = TRUE THEN
    OUTPUT "Number found at index: ", INDEX
ELSE
    OUTPUT "Number not found!"
ENDIF
```

Q25) Write a **pseudocode** to declare an array myList to store each character of myData in a separate element

```
DECLARE X : INTEGER
X ← LENGTH(myData)
DECLARE myList : ARRAY[1: X] OF CHAR
DECLARE INDEX : INTEGER
FOR INDEX ← 1 TO X
        myList[INDEX] ← MID(myData, INDEX, 1)
NEXT INDEX
```

Q26) Write a **pseudocode** for a procedure CountID() it will take StudentID as a parameter and output number of times the entered StudentID appeared in an array myList which contains 10 string values

```
PROCEDURE CountID(StudentID : STRING)
        DECLARE Count, INDEX : INTEGER
        Count ← 0
        FOR INDEX ← 1 TO 10
                IF StudentID = myList[INDEX] THEN
                        COUNT ← COUNT + 1
                ENDIF
        NEXT INDEX
        OUTPUT Count
ENDPROCEDURE
```

Q27) Write a **pseudocode** for a procedure GetLargest() it will take two integer values as parameters (Assume both will be different) and will output the largest one.

```
PROCEDURE GetLargest(N1 : INTEGER, N2 : INTEGER)
        IF N1 > N2 THEN
                        OUTPUT N1
        ELSE
                        OUTPUT N2
        ENDIF
ENDPROCEDURE
```

Bubble Sort:

```
Max ← 5
REPEAT
        SWAP ← FALSE
        FOR INDEX ← 1 TO (MAX – 1)
                IF myList [INDEX] > myList[INDEX + 1] THEN
                        Temp ← myList[INDEX]
                        myList[INDEX] ← myList[INDEX + 1]
                        myList[INDEX + 1] ← Temp
                        SWAP ← TRUE
                ENDIF
        NEXT INDEX
        Max ← Max – 1
UNTIL SWAP = FALSE  OR MAX = 1
```

Question No. 6

A 1D array `Data` of type integer contains 200 elements. Each element has a unique value.

An algorithm is required to search for the largest value and output it.

Describe the steps that the algorithm should perform.

Do **not** include pseudocode statements in your answer.

Question	Answer	Marks
3	One mark per point (**Max 5**): 1 Declare a variable / an integer `Max` 2 Assign value of first / any element to `Max` 3 Set up a loop to repeat 200 times / from start to end of array 4 Use the loop counter as the array index 5 If value of current element is greater than `Max`... 6 ...then assign value to `Max` 7 After the loop, Output `Max`	**5**

```
DECLARE MAX : INTEGER
MAX ← Data[1]
FOR INDEX ← 2 TO 200
        IF Data[INDEX] > MAX THEN MAX ← Data[INDEX]
NEXT INDEX
OUTPUT Max
```

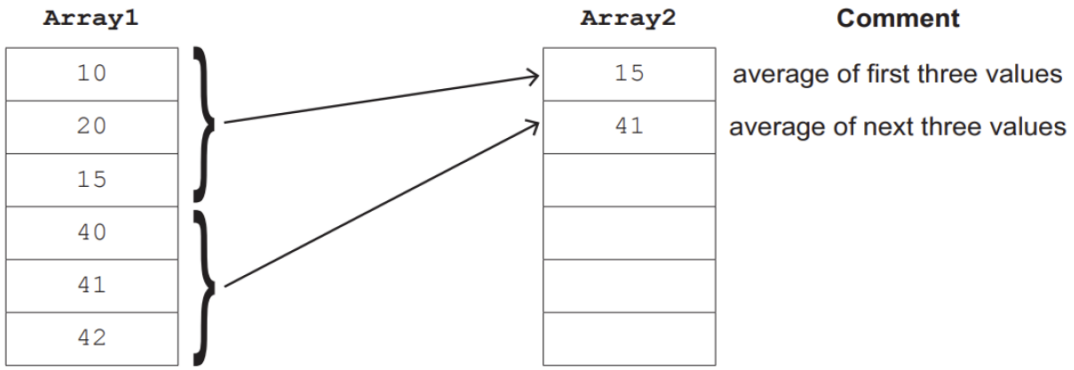
Question No. 4

A program uses two 1D arrays of type integer. `Array1` contains 600 elements and `Array2` contains 200 elements.

`Array1` contains sample values read from a sensor. The sensor always takes three consecutive samples and all of these values are stored in `Array1`.

A procedure `Summarise()` will calculate the average of three consecutive values from `Array1` and write the result to `Array2`. This will be repeated for all values in `Array1`.

The diagram below illustrates the process for the first six entries in `Array1`.



Write pseudocode for the procedure `Summarise()`.

```
PROCEDURE Summarise()
   DECLARE Value : REAL
   DECLARE IxA, IxB : INTEGER // Index variables

   IxB ← 1

   FOR IxA ← 1 TO 598 STEP 3
      Value ← Array1[IxA] + Array1[IxA + 1] + Array1[IxA + 2]
      Value ← Value / 3
      Array2[IxB] ← INT(Value)
      IxB ← IxB + 1
   NEXT IxA

ENDPROCEDURE
```

Question No. 12

(a) A student is learning about arrays.

She wants to write a program to:

- declare a 1D array `RNum` of 100 elements of type `INTEGER`
- assign each element a random value in the range 1 to 200 inclusive
- count and output how many numbers generated were between 66 and 173 inclusive.

(i) Write pseudocode to represent the algorithm.

```
DECLARE RNum : ARRAY[1:100] OF INTEGER
DECLARE Index, Count : INTEGER

Count ← 0
FOR Index ← 1 TO 100
   RNum[Index] ← INT(RAND(200)) + 1
   IF RNum[Index] >= 66 AND RNum[Index] <= 173 THEN
      Count ← Count + 1
   ENDIF
NEXT Index
OUTPUT Count
```

7 A teacher is designing a program to perform simple syntax checks on programs written by students.

Two global 1D arrays are used to store the syntax error data. Both arrays contain 500 elements.

- Array `ErrCode` contains integer values that represent an error number in the range 1 to 800.
- Array `ErrText` contains string values that represent an error description.

The following diagram shows an example of the arrays.

Index	ErrCode	ErrText
1	10	"Invalid identifier name"
2	20	"Bracket mismatch"
3	50	"Undeclared variable"
4	60	"Type mismatch in assignment"
…		
500	999	<Undefined>

Note:

- There may be less than 500 error numbers so corresponding elements in both arrays may be unused. Unused elements in `ErrCode` have the value `999`. The value of unused elements in `ErrText` is undefined.
- Values in the `ErrCode` array are stored in ascending order but not all values may be present, for example, there may be no error code `31`.

The teacher has defined two program modules as follows:

Module	Description
`OutputError()`	• takes two parameters as integers: ○ a line number in the student's program ○ an error number • searches for the error number in the `ErrCode` array: ○ if found, outputs the corresponding error description and the line number, for example: `"Bracket mismatch on line 34"` ○ if not found, outputs the line number and a warning, for example: `"Unknown error on line 34"`
`SortArrays()`	sorts the arrays into ascending order of `ErrCode`

(a) Write **efficient** pseudocode for module `OutputError()`.

```
PROCEDURE OutputError(LineNum, ErrNum : INTEGER)
  DECLARE Index : INTEGER

  Index ← 0

  // Search until ErrNum found OR not present OR end of
array

  REPEAT
     Index ← Index + 1
  UNTIL ErrCode[Index] >= ErrNum OR Index = 500

  IF ErrCode[Index] = ErrNum THEN
     OUTPUT ErrText[Index], " on line ", LineNum
//Found
  ELSE
     OUTPUT "Unknown error on line ", LineNum     //Not
found
  ENDIF

ENDPROCEURE
```

Programming

Question No. 12

A programming language has string functions equivalent to those given in the **insert**.

The language includes a `LEFT()` and a `RIGHT()` function, but it does **not** have a `MID()` function.

(a) Write pseudocode for an algorithm to implement your own version of the `MID()` function which will operate in the same way as that shown in the **insert**.

Do **not** use the `MID()` function given in the **insert**, but you may use any of the other functions.

Assume that the values passed to the function will be correct.

```
FUNCTION MID(InString : STRING, Start, Num : INTEGER)
             RETURNS STRING
   DECLARE MidString : STRING
   DECLARE InStringLen : INTEGER

   InStringLen ← LENGTH(InString)

   // solution for RIGHT() then LEFT()
   MidString ← RIGHT(InString, InStringLen - Start + 1)
   MidString ← LEFT(MidString, Num)

   // alternative solution for LEFT() then RIGHT()
   MidString ← LEFT(InString, Start + Num - 1)
   MidString ← RIGHT(MidString, Num)


   RETURN MidString
ENDFUNCTION
```

Programming

Question No. 10

A string is a palindrome if it reads the same forwards as backwards.

The following strings are examples of palindromes:

"Racecar"
"madam"
"12344321"

Upper-case and lower-case characters need to be treated the same. For example, 'A' is equivalent to 'a'.

A function `IsPalindrome()` will take a string parameter. The function will return `TRUE` if the string is a palindrome and will return `FALSE` if the string is not a palindrome.

Write pseudocode for `IsPalindrome()`.

```
FUNCTION IsPalindrome(InString : STRING) RETURNS BOOLEAN
  DECLARE IsPal : BOOLEAN
  DECLARE Index, Num : INTEGER
  DECLARE CharA, CharB : CHAR

  IsPal ← TRUE
  Index ← 1

  Num ← INT(LENGTH(InString) / 2)

  WHILE Index <= Num AND IsPal = TRUE
     CharA ← MID(InString, Index, 1)
     CharB ← MID(Instring, LENGTH(Instring) - Index + 1,
                 1)
     IF UCASE(CharA) <> UCASE(CharB) THEN
        IsPal ← FALSE // RETURN FALSE
     ENDIF
     Index ← Index + 1
  ENDWHILE

  RETURN IsPal // RETURN TRUE

ENDFUNCTION
```

User-Defined Functions

Q28) Write a **pseudocode** for a function GetLargest() that will take two integer values as parameter and return the largest one

```
FUNCTION GetLargest(N1 : INTEGER, N2 : INTEGER) RETURNS INTEGER
        IF N1 > N2 THEN
                RETURN N1
        ELSE
                RETURN N2
        ENDIF
ENDFUNCTION
```

OR

```
FUNCTION GetLargest(N1 : INTEGER, N2 : INTEGER) RETURNS INTEGER
        DECLARE Largest : INTEGER
        IF N1 > N2 THEN
                Largest ← N1
        ELSE
                Largest ← N2
        ENDIF
        RETURN Largest
ENDFUNCTION
```

Q29) Write a **pseudocode** for a function CheckEqual(). It will take two integer values as parameter and return true if both are same otherwise it will return false

```
FUNCTION CheckEqual(N1 : INTEGER, N2 : INTEGER) RETURNS BOOLEAN
        IF N1 = N2 THEN
                RETURN TRUE
        ELSE
                RETURN FALSE
        ENDIF
ENDFUNCTION
```

OR

```
FUNCTION CheckEqual(N1 : INTEGER, N2 : INTEGER) RETURNS BOOLEAN
        DECLARE IsEqual : BOOLEAN
        IF N1 = N2 THEN
                IsEqual ← TRUE
        ELSE
                IsEqual ← FALSE
        ENDIF
ENDFUNCTION
```

Q30) Write a **pseudocode** for a function GetCount(). It will take a string and a character as parameters and returns the number of repetitions of the character in the string.

```
FUNCTION GetCount(MyData : STRING, Letter : CHAR) RETURNS INTEGER
    DECLARE Count, Index : INTEGER
    Count ← 0
    FOR Index ← 1 TO  LENGTH(MyData)
        IF MID(MyData, Index, 1) = Letter THEN
            Count ← Count + 1
        ENDIF
    NEXT INDEX
    RETURN Count
ENDFUNCTION
```

Q31) Write a **pseudocode** for a function GetRemarks(). It will take percentage in two decimal places as a parameter and return "PASS" if percentage is 60 and above otherwise return "FAIL"

```
FUNCTION GetRemarks(Percentage: REAL) RETURNS STRING
    IF Percentage >= 60 THEN
        RETURN "PASS"
    ELSE
        RETURN "FAIL"
    ENDIF
ENDFUNCTION
```

- **You can also use variable to return like in Q28 & Q29.**
- **Just for curiosity if you want to check the decimal places of the percentage you can make and use a function, it will first parse the data to string then it will find '.'(decimal) in the string using a loop then using it's index and length right function you can separate how much characters are their and you have found the number of decimal places.**
- **If you want to round a number to x number of decimal places then you can use some of the logic above and use the index after your required decimal places you want to round up to and parse it to integer and check if it is equal or greater than 5 so increase the required decimal place value otherwise don't change it.**
- **A function ends where the return statement is called**

RECORD DATATYPE

Records are composite data types formed by the inclusion of several related items that may be of different data types. This allows a programmer to refer to these items using the same identifier, enabling a structured approach to using related items. A record will contain a fixed number of items. For example, a record for a book could include title, author, publisher, number of pages, and whether it is fiction or non-fiction.

A record data type is one example of a composite user-defined data type. A composite data type references other existing data types when it is defined. A composite data type must be defined before it can be used. Any data type not provided by a programming language must be defined before it can be used.

In pseudocode, a record data type definition takes the following form:

```
TYPE <TypeName>
        DECLARE <identifier> : <data type>
        DECLARE <identifier> : <data type>
        DECLARE <identifier> : <data type>
        : :
        : :
ENDTYPE
```

Example:

```
TYPE Students
        DECLARE StudentID : STRING
        DECLARE Name : STRING
        DECLARE Age : INTEGER
        DECLARE GENDER : CHAR
ENDTYPE

DECLARE MyStudent : Students
MyStudent.StudentID ← "MR1001"
MyStudent.Name ← "Ali"
MyStudent.Age ← 17
MyStudent.Gender ← "M"

DECLARE MyList : ARRAY[1:5] OF Students

MyList[5].StudentID ← "MR1005"
MyList[5].Name ← "Muneeb"
MyList[5].Age ← 18
MyList[5].Gender ← "M"
```

Question No. 8

A company keeps details of its product items in a 1D array, `Stock`. The array consists of 1000 elements of type `StockItem`.

The record fields of `StockItem` are:

Field	Typical value
`ProductCode`	`"BGR24-C"`
`Price`	`102.76`
`NumberInStock`	`15`

(a) Write pseudocode to declare the record structure `StockItem`.

```
TYPE StockItem
        DECLARE ProductCode : STRING
        DECLARE Price : REAL
        DECLARE NumberInStock : INTEGER
ENDTYPE
```

(b) Write pseudocode to declare the `Stock` array.

```
DECLARE Stock : ARRAY [1:1000] OF StockItem
```

(c) Write pseudocode to modify the values to element 20 as follows:

- set the price to 105.99
- increase the number in stock by 12

```
Stock[20].Price ← 105.99
Stock[20].NumberInStock ← Stock[20].NumberInStock + 12
```

(d) A stock report program is developed.

Write pseudocode to output the information for each stock item that has a price of at least 100.

Output the information as follows:

```
Product Code: BGR24-C Number in Stock: 15
```

```
DECLARE n : INTEGER
FOR n ← 1 TO 1000
        IF Stock[n].Price >= 100
                THEN
                        OUTPUT "ProductCode: " & Stock[n].ProductCode &
                        " Number in Stock: " & Stock[n].NumberInStock
        ENDIF
NEXT n
```

Question No. 9

A programmer is writing a program to help manage clubs in a school.

Data will be stored about each student in the school and each student may join up to three clubs.

The data will be held in a record structure of type `Student`.

The programmer has started to define the fields that will be needed as shown in the following table.

Field	Typical value	Comment
`StudentID`	`"CF1234"`	Unique to each student
`Email`	`"Carmen47@xyzmail.com"`	Contains letters, numbers and certain symbols
`Club_1`	`1`	Any value in the range 1 to 99 inclusive
`Club_2`	`14`	Any value in the range 1 to 99 inclusive
`Club_3`	`27`	Any value in the range 1 to 99 inclusive

(a) (i) Write pseudocode to declare the record structure for type `Student`.

```
TYPE Student
    DECLARE StudentID : STRING
    DECLARE Email : STRING
    DECLARE Club_1 : INTEGER
    DECLARE Club_2 : INTEGER
    DECLARE Club_3 : INTEGER
ENDTYPE
```

(ii) A 1D array `Membership` containing 3000 elements will be used to store the student data.

Write pseudocode to declare the `Membership` array.

```
DECLARE Membership : ARRAY [1:3000] OF Student
```

(iii) Some of the elements of the array will be unused.

Give an **appropriate** way of indicating an unused array element.

Assign an empty string to the StudentID field or Email field

(iv) Some students are members of less than three clubs.

State **one** way of indicating an unused club field.

A number outside the range from 1 to 99

(b) A procedure `GetIDs()` will:

- prompt and input the number of a club
- output the `StudentID` of all the students who are members of that club
- output a count of all students in the given club.

Write pseudocode for the procedure `GetIDs()`.

```
PROCEDURE GetIDs()
    DECLARE Index, ThisClub, Count : INTEGER

    OUTPUT "Please Input Club Number: "
    INPUT ThisClub

    Count ← 0

    FOR Index ← 1 TO 3000
        IF Membership[Index].Club_1 = ThisClub OR
           Membership[Index].Club_2 = ThisClub OR
           Membership[Index].Club_3 = ThisClub THEN
            Count ← Count + 1
            OUTPUT Membership[Index].StudentID
        ENDIF
    NEXT Index

    OUTPUT "There are ", Count, " Students in the club"
ENDPROCEDURE
```

Question No. 10

A program will store attendance data about each employee of a company.

The data will be held in a record structure of type `Employee`. The fields that will be needed are as shown:

Field	Typical value	Comment
`EmployeeNumber`	`123`	A numeric value starting from 1
`Name`	`"Smith,Eric"`	Format: <last name>','<first name>
`Department`	`"1B"`	May contain letters and numbers
`Born`	`13/02/2006`	Must not be before 04/02/1957
`Attendance`	`97.40`	Represents a percentage

(a) **(i)** Write pseudocode to declare the record structure for type `Employee`.

```
TYPE Employee
    DECLARE EmployeeNumber : INTEGER
    DECLARE Name : STRING
    DECLARE Department : STRING
    DECLARE Born : DATE
    DECLARE Attendance : REAL
ENDTYPE
```

(ii) A 1D array `Staff` containing 500 elements will be used to store the employee records.

Write pseudocode to declare the `Staff` array.

```
DECLARE Staff : ARRAY [1:500] OF Employee
```

(b) There may be more records in the array than there are employees in the company. In this case, some records of the array will be unused.

(i) State why it is good practice to have a standard way to indicate unused array elements.

- So that unused elements may be recognized while processing / searching
- Otherwise the element may contain old / unexpected data

(ii) Give **one** way of indicating an unused record in the `Staff` array.

An EmployeeNumber field < 1 can be used

(c) A procedure `Absentees()` will output the `EmployeeNumber` and the `Name` of all employees who have an `Attendance` value of 90.00 or less.

Write pseudocode for the procedure `Absentees()`.

Assume that the `Staff` array is global.

```
PROCEDURE Absentees()
    DECLARE Index : INTEGER
    FOR Index ← 1 TO 500
        IF Staff[Index].EmployeeNumber > 1 THEN          // not empty
            IF Staff[Index].Attendance <= 90 THEN
                OUTPUT Staff[Index].EmployeeNumber
                OUTPUT Staff[Index].Name
            ENDIF
        ENDIF
    NEXT Index
ENDPROCEDURE
```

BYVAL & BYREF

There are two methods of passing a parameter to a procedure: **by value** and **by reference**. When a parameter is passed by value, if a variable is used, the value of that variable cannot be changed within the procedure. When a parameter is passed by reference the value of the variable passed as the parameter can be changed by the procedure.

A procedure is by value at default so there is no need to mention a **by value** procedure but **by reference** needs to be mentioned.

A procedure with parameters passed by reference can be defined in pseudocode as follows:

```
PROCEDURE <identifier> (BYREF <parameter1>:<datatype>, <parameter2>:<datatype>...)
        <statements>
ENDPROCEDURE
```

Ex 1:

```
PROCEDURE celsius(BYREF temperature : REAL)
        temperature ← (temperature – 32) / 1.8
ENDPROCEDURE

CALL celsius(41.6)
```

Ex 2:

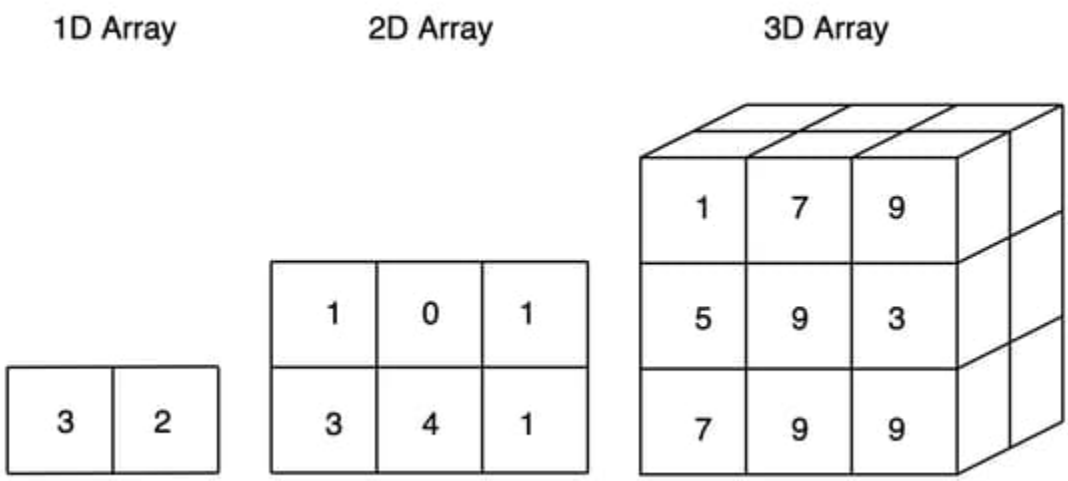
```
PROCEDURE UpdateMarks(BYREF Marks : INTEGER)
        IF Marks = 49 THEN
                Marks ← 50
        ENDIF
ENDPROCEDURE

PROCEDURE GetRemark(BYVAL Marks : INTEGER)
        IF Marks >= 50 THEN
                Remarks ← "PASS"
        ELSE
                Remarks ← "FAIL"
        ENDIF
ENDPROCEDURE
```

- In the procedure GetRemark() 'BYVAL' is useless as the procedure is by default by value

2D Array

- Array can be multidimensional



- 2D Array is like a table. It is defined using rows & columns.

DECLARE <Array Name> : ARRAY [RowIndex, ColumnIndex] OF <Datatype>

for e.g :- DECLARE MyList : ARRAY [1:5, 1:5] OF INTEGER

MyList[]

	1	2	3	4	5
1					
2					
3					
4					
5					

```
FOR Row ← 1 TO 5
    FOR Col ← 1 TO 5
        MyList[Row, Col] ← -1
    NEXT Col
NEXT Row
```

MyList[]

	1	2	3	4	5
1	-1	-1	-1	-1	-1
2	-1	-1	-1	-1	-1
3	-1	(3,2) -1	-1	-1	-1
4	-1	-1	-1	-1	(4,5) -1
5	(5,1) -1	-1	-1	-1	-1

Zuhaib Abdul Khalique | zuhaib.khaliq@gmail.com

Q32) Write a **pseudocode** to flip all the 0's with 1's and 1's with zero in MyList[ ]

MyList[ ]

1	0	1	0	1
0	1	0	1	0
1	0	1	0	1
0	1	0	1	0
1	0	1	0	1

```
DECLARE Row, Col : INTEGER
FOR Row ← 1 TO 5
        FOR Col ← 1 TO 5
                IF MyList[Row,Col] = 0
                        THEN
                                MyList[Row,Col] ← 1
                        ELSE
                                MyList[Row,Col] ← 0
                ENDIF
        NEXT Col
NEXT Row
```

Q33) Write a **pseudocode** to declare and initialise an Array MyList[] with 5 Rows and 5 Columns. Initialise it with a random integer number in between 1 and 6 inclusive, output number of times 6 is stored in Array MyList[].

Here is an extract from insert:

`INT(x : REAL) RETURNS INTEGER` returns the integer part of `x` Example: `INT(27.5415)` returns `27`
`RAND(x : INTEGER) RETURNS REAL` returns a real number in the range `0` to `x` (**not** inclusive of `x`). Example: `RAND(87)` may return `35.43`

```
DECLARE MyList[1:5,1:5] OF INTEGER
Count ← 0
FOR Row ← 1 TO 5
        FOR Col  ← 1 TO 5
                MyList[Row,Col] ← INT(RAND(6) + 1)
                IF MyList[Row,Col] = 6 THEN
                        Count ← Count + 1
                ENDIF
        NEXT Col
NEXT Row
OUTPUT Count
```

Q34) Write a user-defined function **GetMarks()** it will take StudentID as a parameter and return the corresponding marks if it is found otherwise return -1

Array MyList() contains 100 unique StudentIDs along with their marks

MyList[]

	1	2
1	1005	76
2	1001	84
3	1004	98
4	1008	90
5	1010	60
6	1999	57
7	?	?
100	1921	85

```
FUNCTION GetMarks(StudentID : INTEGER) RETURNS INTEGER
    DECLARE Index, Marks : INTEGER
    DECLARE Found : BOOLEAN
    Index ← 1
    Found ← FALSE
    Marks ← -1
    REPEAT
        IF StudentID = MyList[Index,1]
            THEN
                Marks ← MyList[Index,2]
                Found ← TRUE
            ELSE
                Index ← Index + 1
        ENDIF
    UNTIL Index > 100 OR Found = TRUE
    RETURN Marks
ENDFUNCTION
```

OR

```
FUNCTION GetMarks(StudentID : INTEGER) RETURNS INTEGER
    DECLARE Index : INTEGER
    Index ← 1
    REPEAT
        IF StudentID = MyList[Index, 1]
            THEN
                RETURN MyList[Index,2]
            ELSE
                Index ← Index + 1
        ENDIF
    UNTIL Index > 100
    RETURN -1
ENDFUNCTION
```

Q35) Write a pseudocode for a procedure **BubbleSort()** to sort an array **MyList[]** in descending order on Marks

MyList[]

	1	2
1	1005	76
2	1001	84
3	1004	98
4	1008	90
5	1010	60
6	1999	57
7	?	?
	⋮	⋮
100	1921	85

```
PROCEDURE BubbleSort()
        DECLARE Max, Index, Temp : INTEGER
        DECLARE Swap : BOOLEAN

        Max ← 100
        REPEAT
                Swap ← FALSE
                FOR Index ← 1 TO Max-1
                        IF MyList[Index,2] < MyList[Index+1,2] THEN

                                Temp ← MyList[Index,1]
                                MyList[Index,1] ← MyList[Index+1,1]
                                MyList[Index+1,1] ← Temp

                                Temp ← MyList[Index,2]
                                MyList[Index,2] ← MyList[Index+1,2]
                                MyList[Index+1,2] ← Temp

                                Swap ← TRUE
                        ENDIF
                NEXT Index
                Max ← Max-1
        UNTIL Swap = FALSE OR Max = 1
ENDPROCEDURE
```

Q36) **6** A procedure `Square()` will take an integer value in the range 1 to 9 as a parameter and output a number square.

The boundary of a number square is made up of the character representing the parameter value. The inside of the number square is made up of the asterisk character (`*`).

Parameter value	**1**	**2**	**3**	**4**	**...**	**9**
Output	1	22 22	333 3*3 333	4444 4**4 4**4 4444	...	999999999 9*******9 9*******9 9*******9 9*******9 9*******9 9*******9 9*******9 999999999

The pseudocode `OUTPUT` command starts each output on a new line. For example, the following three `OUTPUT` statements would result in the outputs as shown:

```
OUTPUT "Hello"
OUTPUT "ginger"
OUTPUT "cat"
```

Resulting output:

```
Hello
ginger
cat
```

```
PROCEDURE Square(Num : INTEGER)
    DECLARE Index : INTEGER
    DECLARE MyChar : CHAR
    DECLARE Str1, Str2 : STRING
    CONSTANT Asterik = "*"
    MyChar ← NUM_TO_STR(Num)
    Str1 ← ""
    FOR Index ← 1 TO Num
        Str1 ← Str1 & MyChar
    NEXT Index
    IF Num = 1 THEN
        OUTPUT MyChar
    ELSE IF Num = 2 THEN
        OUTPUT Str1
        OUTPUT Str1
    ELSE
        Str2 ← MyChar
        FOR Index ← 1 TO Num - 2
            Str2 ← Str2 & Asterik
        NEXT Index
        Str2 ← Str2 & MyChar
        OUTPUT Str1
        FOR Index ← 1 TO Num – 2
            OUTPUT Str2
        NEXT Index
        OUTPUT Str1
    ENDIF
ENDPROCEDURE
```

Q37) Write a **pseudocode** for tic-tac-toe game

	1	2	3
1	X	O	X
2	O	X	O
3	X	O	X

- There are total 8 possible combinations for winning → 3 Rows, 3 Columns and 2 Diagonals

```
DECLARE Board : ARRAY[1:3,1:3] OF CHAR
DECLARE PlayerTurn, Winner : CHAR
DECLARE Row, Col, Moves : INTEGER
DECLARE Won : BOOLEAN

FUNCTION CheckWin() RETURNS BOOLEAN
        FOR Row ← 1 TO 3
                IF (Board[Row, 1] = Board[Row,2] AND Board[Row,2] = Board[Row,3]) AND
                   (Board[Row,1] = 'X' OR Board[Row,1] = 'O')
                        THEN
                                Winner ← Board[Row,1]
                                RETURN TRUE
                ENDIF
        NEXT Row

        FOR Col ← 1 TO 3
                IF (Board[1,Col] = Board[2,Col] AND Board[2,Col] = Board[3,Col]) AND
                   (Board[1,Col] = 'X' OR Board[1,Col] = 'O')
                        THEN
                                Winner ← Board[1,Col]
                                RETURN TRUE
                ENDIF
        NEXT Col

        IF (Board[1,1] = Board[2,2] AND Board[2,2] = Board[3,3]) AND
           (Board[1,1] = 'X' OR Board[1,1] = 'O')
                THEN
                        Winner ← Board[1,1]
                        RETURN TRUE
        ENDIF

        IF (Board[1,3] = Board[2,2] AND Board[2,2] = Board[3,1]) AND
           (Board[1,3] = 'X' OR Board[1,3] = 'O')
                THEN
                        Winner ← Board[1,3]
                        RETURN TRUE
        ENDIF
```

```
        RETURN FALSE
ENDFUNCTION
PlayerTurn ← 'X'
Moves ← 0
Won ← FALSE
Winner ← ''

REPEAT
        OUTPUT "Player ", PlayerTurn, " enter row (1-3): "
        INPUT Row
        OUTPUT "Player ", PlayerTurn, " enter column (1-3): "
        INPUT Col

        IF (Board[Row,Col] <> 'X' AND Board[Row,Col] <> 'O') AND
           (Row <= 3 AND Row >= 1) AND
           (Col <= 3 AND Col >= 1)
                THEN
                        Board[Row,Col] ← PlayerTurn
                        Moves ← Moves + 1
                        Won ← CheckWin()

                        IF Won
                                THEN
                                        OUTPUT "Player ", Winner, " wins!"
                        ENDIF

                        IF PlayerTurn = 'X'
                                THEN
                                        PlayerTurn  ← 'O'
                                ELSE
                                        PlayerTurn ← 'X'
                        ENDIF
                ELSE
                        IF (Row < 1 OR Row > 3 OR Col < 1 OR Col > 3) THEN
                                OUTPUT "Invalid! Please enter a row and column between 1 and 3."
                        ELSE
                                OUTPUT "Cell already occupied! Please choose another cell."
                        ENDIF
        ENDIF
UNTIL Moves = 9 OR Won

IF Won = FALSE
        THEN
                OUTPUT "It's a draw!"
ENDIF
```

- A procedure to initialise array and a procedure to display the contents of the array after every turn could have been also used

Random Function

```
RAND(x : INTEGER) RETURNS REAL
```

returns a real number in the range `0` to `x` (**not** inclusive of `x`).
Example: `RAND(87)` could return `35.430729`

Module	Description
`RandomChar()`	• Generates a single random character from within one of the following ranges: ○ 'a' to 'z' ○ 'A' to 'Z' ○ '0' to '9' • Returns the character

For reference, relevant ASCII values are as follows:

Character range	ASCII range
'a' to 'z'	97 to 122
'A' to 'Z'	65 to 90
'0' to '9'	48 to 57

(a) Write pseudocode for module `RandomChar()`.

You may wish to refer to the **insert** for a description of the `CHR()` function. Other functions may also be required.

```
FUNCTION RandomChar() RETURNS CHAR
    DECLARE MyChar : CHAR
    DECLARE MyInt : INTEGER

    MyInt ← INT(RAND(3))

    IF MyInt = 0 THEN
        MyChar ← CHR(INT(RAND(10)) + 48)
    ELSE IF MyInt = 1 THEN
        MyChar ← CHR(INT(RAND(26)) + 65)
    ELSE
        MyChar ← CHR(INT(RAND(26)) + 97)
    ENDIF

    RETURN MyChar
ENDFUNCTION
```

Programming Errors

1. **Syntax Error** – Errors in the grammar of a source program
2. **Logic Error** – Errors in the logic of a program
3. **Run Time Error** – Errors that happen when the program is e

Ex:

1. DECLARE N1, N2, Sum : INTEGER
2. INPUT N1
3. INPT N2
4. Sum ← N1 – N2
5. OUTPUT Sum

In Line 3 there is a syntax error the code should be 'INPUT N2'

In Line 4 there is a logic error as sum will be N1 + N2 not N1 – N2

An example of a runtime error will be when we input 'ten' in N1 instead of '10'. This exception wasn't considered in the code hence it will produce an error.

Another example of runtime error is that a code divides two values and the divisor is input as zero producing a division by zero error

Test Data

Types of test data:

1. **Normal** – data that is accepted by a program and is used to show that program is working as expected
2. **Abnormal** – data that should be rejected by a program as it is unsuitable or could cause problems
3. **Extreme** – data that is on the limit of that accepted by a program → Upper Limit & Lower Limit
4. **Boundary** – data that is on the limit of that accepted by a program or data that is just outside the limit of that rejected by a program → Upper Limit and value just above it or Lower Limit and value just below it.

Ex: A program accepts age between 15 to 25 inclusive of them

Normal Data example: 18 or 20 or 22
Abnormal Data example: 10 or 30 or 28
Extreme Data example: 15 or 25
Boundary Data example: 14,15 or 25,26